

MISAKA

耐量子 BlockDAG、Header-v5 Compute Set Registry、モデル非依存の検証可能 LLM Work

2026 年 8 月 1 日

<https://github.com/MISAKA-BTC/misakas> • Explorer: <https://misakachain.com>

要旨

MISAKA は *rusty-kaspa* を基盤とする独立した PQ-only Layer-1 であり、GHOSTDAG BlockDAG と UTXO ledger を維持しつつ、取引認可を ML-DSA-87、consensus identity を 64-byte keyed BLAKE2b-512、UTXO commitment を additive lattice accumulator へ移行する。本版は、Version 3 の概念仕様を、2026-08-01 時点の提出コード、ADR、runbook、network parameters から再構成した code-derived specification である。

Version 4 の中心は、単一モデルを consensus code に固定する方式から、モデルを登録データとして扱う *Model-Agnostic Compute Set Registry* への移行である。各 Compute Set は、immutable descriptor、source DAA で解決される mutable policy、全 active set を一括再記述する atomic allocation plan の三層に分離される。Header v5 は `compute_set_id`、`compute_policy_id`、`allocation_plan_id` を hash preimage に含めるため、将来のモデル追加、経済設定変更、停止を header schema の追加変更なしで表現できる。

公開ネットワークは v5 re-genesis の `testnet-20` である。PQ-only consensus、DNS-style two-dimensional finality、PALW audited-compute lane、algo-4 acceptance、anti-spam accumulator、Compute Set registry は genesis から有効である。一方、mainnet は未ローンチであり、testnet の MSK と MTP は価値、換金性、配布約束を持たない。EVM implementation は repository に存在するが、`testnet-20` preset では activation fence が閉じている。

本書は「実装済み」「ゲート中」「設計のみ」を区別する。現 snapshot では、descriptor/policy/plan の content-addressed store、fork-local view、ML-DSA-87 validator-quorum activation certificate、per-set difficulty、historical credit resolution、permissionless snapshot authentication、nullifier prune/replay E2E が実装されている。他方、registry governance action の認可、proposal bond の value/lock 検証、header reference と fork-local governing view の同一性、activation certificate の content identity による複数版保存、Compute IR payload の proposal admission 統合、set-centric audit certificate の実 replay verifier、PCPB/fraud wiring、外部二者再現は release gate として残る。したがって本書は、LLM の回答が真実であること、特定 GPU が物理的に計算したこと、mainnet security、または経済価値を主張しない。

Status notice. 本書は 2026-08-01 の提出 code snapshot に基づく research specification である。`testnet-20` は no-value public testnet であり、mainnet 稼働や経済価値を主張しない。

1 はじめに

1.1 本書の目的

MISAKA を動機づける圧力は、暗号、並列 consensus、計算資源の三つである。第一に、classical elliptic-curve authorization は長期の量子 adversary に対して安全余裕を維持しない。第二に、linear chain は propagation delay を orphan pressure に変える一方、BlockDAG は concurrent honest block を保持して ordering へ変換できる。第三に、日常の LLM inference は大きな accelerator capacity を消費するが、単に「GPU を使った」だけでは公開検証可能な work にならない。input secrecy、floating-point divergence、withholding、Sybil identity、再実行費用が残るためである。

Version 1 は PQ UTXO consensus、Hash64 identity、layered PoW、selected-parent EVM を扱った。Version 2 は PALW audited-compute lane を導入した。Version 3 は hardware pool を廃止し

て単一整数正準 compute set へ閉じた。Version 4 はさらに、モデル名と runtime を consensus binary に固定する前提を外し、登録済みの bit-exact computation を versioned registry object として扱う。

1.2 Version 4 の貢献

1. PQ-only UTXO ledger、GHOSTDAG ordering、DNS finality、selected-parent execution lane、PALW を一つの base consensus 上で整理する。
2. Compute Set の identity、economics、allocation を descriptor、policy、plan へ分離する。
3. Header v5 で exact set/policy/plan reference を block identity に束縛する。
4. future model addition を proposal、reproduction、activation certificate、Shadow、Active、allocation plan の手順へ落とす。
5. Compute IR V1 と external runtime plugin により、model-specific native consensus code を避ける。
6. source block が commit した historical record から compute credit と per-set difficulty を再構成する。
7. 実装済み path と未解決 security gate を同じ status matrix に置く。

1.3 Version 3 との関係

V4 は V3 の PQ base、DNS finality、PALW ticket、receipt、checkpoint、DA、audit quorum、block authorization、one-shot nullifier を継承する。主な変更は、V3 の「唯一の正規 compute set」を、V4 の「複数の immutable Compute Set と、その時点の governing policy/plan」へ一般化したことである。

V3 の QW36-INT-W4A8-V1 は、V4 では一つの descriptor 候補として表現できる。Qwen、Llama、MoE、vision model などの表示名自体は consensus identity ではない。identity を構成するのは、artifact、tokenizer、template、preprocessing、decode policy、semantic program、shape/cost table、arithmetic rule、overflow budget、LUT、trace/checkpoint policy、conformance vector、resource limit の content root である。

1.4 証拠方法と snapshot identity

本書は提出された二つの ZIP を対象に静的追跡を行った。snapshot identity は次のとおりである。

評価は、README の宣言だけでなく、header preimage、network params、registry object、virtual processor fold、header-stage validation、mint path、store key、test、ADR の相互参照に基づく。comment と実装が矛盾する場合、activated code path と parameter preset を優先する。

2 STATUS、SCOPE、CLAIM BOUNDARY

2.1 normative vocabulary

本書では、次の語を区別する。

landed / 実装済み

型、validation、store、test が repository に存在する。

accepted / 受理

network parameter が block/transaction path を許可する。

Table 1: 提出 snapshot の identity.

Artifact	SHA-256
Code ZIP	1e51ed3066484984e01515acedb788130c0d1832fecdac2b6750e1abf0aa007c
Whitepaper ZIP	4578a3b081286acfb6ac0a3d7d6c86bf312fe225a6b9604caa99766d2f420b84
Code README.md	844056b1870db3ae78a0d680ffc736727ce4f223bfcfc6c6d9ca30d29bba2332a
V3 Japanese PDF	f50a21bbd5df90b0f8c0b3ffc45e333a64d28f99ff43b71aafe9e1c343e0051b

weighted / 加重

fork choice または reward で非ゼロの economic/work effect を持つ。

public-value ready

permissionless operation、外部監査、soak、economic calibration、incident response を満たす。

mainnet ready

immutable genesis/parameter ceremony、launch governance、production support を含む。

これらは同義ではない。たとえば testnet-20 は PALW algo-4 acceptance と registry が active だが、flat compute work scale は zero であり、個々の Compute Set が自動的に nonzero weight を持つわけではない。また、activation certificate verifier が実装されていても、proposal/policy/plan/halt の governance authorization が無ければ、permissionless economic control は完成しない。

2.2 ネットワーク状態

2.3 主張すること

MISAKA が code level で主張する範囲は、以下である。

- PQ consensus mode における transaction authorization は ML-DSA-87 であり、legacy secp256k1/Schnorr/ECDSA/P2SH path は標準 consensus path から除外される。
- block hash、txid、Merkle root、UTXO commitment identity などは 64-byte Hash64 を用いる。
- Compute Set descriptor、policy、allocation plan は canonical Borsh bytes と domain-separated keyed BLAKE2b-512 から identity を導出する。
- Header v5 は PALW block が名指す set/policy/plan reference を block hash に束縛する。
- activation certificate は selected-parent DNS bond view に対して validator vote、ML-DSA-87 signature、stake tally、2/3 quorum を再計算する。
- source block の compute credit は、その block が commit した exact policy/plan record から再計算できる。

2.4 主張しないこと

非主張

- LLM output が事実、正解、安全、無害であること。
- receipt が特定の物理 GPU、所在地、電力源、人物による実行を証明すること。
- private local inference が、公開 DA、独立再実行、audit を経ずに mint 対象になること。
- testnet MSK、MTP、provider receipt が金融資産、債権、換金権、airdrop entitlement であること。
- mainnet が稼働中、または現在の testnet parameter がそのまま mainnet parameter になること。
- 型定義だけ存在する PalwComputeSetAuditCertificateV1 が、現時点で full replay と auditor signature を consensus verification していること。

3 予備知識と記法

3.1 hash と content identity

$\text{Hash}_{64}(d, x) = \text{BLAKE2b-512}(\text{key} = d, \text{input} = x)$.

d は domain separator、 x は canonical bytes である。Hash64 は 64 byte。content-derived ID は proposer が任意に選ばず、object の canonical Borsh encoding から導出する。

3.2 DAA と selected-parent coordinate

$\text{DAA}(B)$ は block B の DAA score、 $\text{SP}(B)$ は selected parent、 V_B は B の branch における fork-local registry view とする。policy/plan の effective condition は wall clock ではなく DAA coordinate で判定する。

3.3 work notation

H_B は real hash work、 C_B は policy で正規化された compute work、 E_B は component work cap 後の effective work とする。 V_3 から継承する安全上限は概念的に

$$E_B = H_B + \min(C_B, 4H_B)$$

であり、compute stack が破られても永久 hash floor を無限に置換しない。

3.4 quorum notation

selected validator stake の合計を T 、PASS stake を A とすると、2/3 quorum は overflow を避ける整数比較

$$3A \geq 2T$$

で判定する。certificate 内の tally は信頼せず、embedded vote と selected-parent bond view から再計算する。

4 SYSTEM OVERVIEW

4.1 protocol stack

4.2 設計目標

- PQ-only base authorization:** classical signature path を fallback として残さない。
- model as data:** node binary にモデル固有 verifier や価格表を hard-code しない。
- historical determinism:** 過去 block の credit は、後日の governance 変更から独立して再計算できる。
- fail closed:** unknown VM、unknown set、missing record、zero share、inactive state を default へ丸めない。
- fork-local governance state:** content record は write-once、適用 view は selected-parent と accepted mergeset から branch ごとに導出する。
- bounded replacement:** compute work は permanent hash floor を完全に消さない。
- separation of claims:** block acceptance、work credit、payment、slashing、mainnet readiness を別 gate とする。

4.3 non-goal

V_4 は、general-purpose zkML、秘密 input の完全非公開推論、hardware remote attestation、モデル出力の真偽 oracle、任意 floating-point runtime の exact equivalence を提供しない。現段階では、公開された canonical data と deterministic integer semantics の再実行可能性を中心に置く。

Table 2: ネットワークと activation 状態。

Network	Header / Genesis	状態	主要意味
testnet-20	v5 / registry genesis	実装済み public testnet	PQ、DNS、PALW、algo-4、anti-spam、registry が genesis-active。public DNS seed、P2P port 26521。
testnet-200	v4 / staging PALW genesis	deprecated	mid-chain DNS threshold change による unsafe replay のため public seeder を削除。旧 ledger operator 向けに compilable。
testnet-10	legacy public preset	retired	compatibility preset。public operation と新規 MTP 対象ではない。
mainnet	constants defined	ゲート中	genesis constants と params は存在するが、launch、endorsement、production support はない。
selected-parent EVM	Header v2 path	ゲート中	implementation は存在するが、testnet-20 の evm_activation_daa_score=u64::MAX。

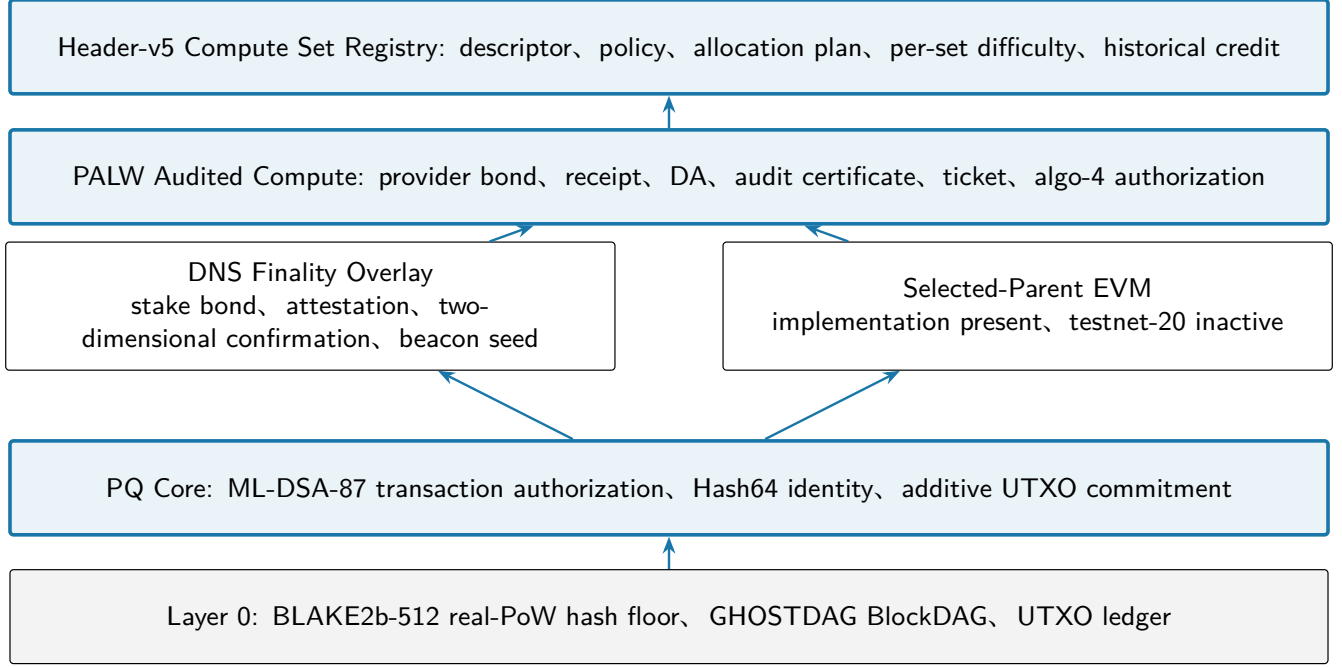


Figure 1: MISAKA V4 の論理スタック。矢印は依存関係を表し、すべてのレーンが同一の activation state にあることを意味しない。

5 耐量子 CRYPTOGRAPHIC CORE

5.1 ML-DSA-87 authorization

PQ enforcement が consensus mode で active な network では、standard transaction authorization は ML-DSA-87 を用いる。script path は fixed-size public-key hash と ML-DSA signature verification を中心に構成し、classical curve を黙って受け入れる compatibility fallback を持たない。PQ claim は base UTXO lane に対するものであり、EVM typed transaction sender recovery の classical ECDSA compatibility と混同してはならない。

5.2 Hash64 identity

block hash、transaction ID、Merkle node、registry object ID、certificate hash、commitment root は 64-byte identity を用いる。domain-separated keyed BLAKE2b-512 は、同じ bytes が別 object type で同一 identity と解釈される cross-domain collision を避ける。

5.3 additive UTXO commitment

UTXO set commitment は insert/remove を可換な加法で更新できる lattice accumulator を用いる。selected-parent EVM bridge が有効化される場合、deposit consumption、credit、withdraw debit、UTXO materialization が同じ commitment coordinate に入る必要がある。bridge accounting の存在は EVM activation を意味しない。

5.4 transport と implementation scope

consensus signature が PQ-only でも、P2P transport、TLS、OS entropy、compiler、dependency supply chain が自動的に PQ にな

るわけではない。V4 の PQ claim は consensus-visible authorization と identity の範囲に限定する。

6 LAYERED POW と GHOSTDAG

6.1 permanent hash floor

MISAKA は real PoW hash lane を permanent floor として保持する。PALW algo-4 block は audited-compute ticket と authorization を使うが、hash floor と component-work cap の上に存在する。GPU inference capacity が fork choice を完全に独占しないよう、compute work の credit は bounded component として扱う。

6.2 GHOSTDAG ordering

concurrent block は DAG に保持され、GHOSTDAG の selected parent、blue score、blue work により ordering へ変換される。registry state は global mutable singleton ではなく、selected-parent view と accepted mergeset effects から block ごとに導出する。この構造が reorg 時の governance state を branch-local に保つ。

6.3 component work

Compute Set policy は compute_work_scale と weight_factor_bps を持つ。source block B の normalized compute work を Q_B とすると、概念上の credit は

$$C_B = \left\lfloor Q_B \cdot \text{compute_work_scale}_B \cdot \frac{\text{weight_factor_bps}_B}{10,000} \right\rfloor$$

である。添字 B は current policy ではなく、source header が commit した historical policy/plan を意味する。

7 DNS FINALITY と CONSENSUS RANDOMNESS

7.1 bond と validator lifecycle

DNS overlay は stake-bonded validator record、epoch attestation、slashing/unbond lifecycle を持つ。testnet-20 は bring-up のため minimum active validator を 1、minimum bond と active stake を 10 MSK に下げるが、quorum の意味は stake-weighted 2/3 のままである。

7.2 two-dimensional confirmation

DNS confirmation は PoW WorkDepth と StakeDepth の二つを満たす。testnet-20 の operator-facing 値は WorkDepth 100、StakeDepth 5000 であり、pov-dependent sink hash ではなく stake-confirmed anchor を finality coordinate として扱う。

7.3 beacon seed

PALW の provider assignment、auditor slate、sample plan、maturity は、commit 後の DNS-confirmed randomness から導出する。役割を commitment 前に選べると、自分に都合のよい replica/auditor を grind できるため、assignment は future beacon へ遅延させる。

7.4 registry activation certificate

Compute Set activation は selected-parent の frozen DNS bond view に束縛する。各 vote は certificate summary、network id、bond outpoint、vote bit を ML-DSA-87 で署名する。verifier は validator set commitment、vote order/uniqueness、signature、active bond、stake tally、quorum を再計算する。

8 HEADER VERSIONING と RE-GENESIS

8.1 header version evolution

各 extension は version gate 未満では hash-invisible かつ zero を要求する。既存 block identity を書き換えずに新 preimage field を導入するためである。v5 genesis は三 reference を zero にする。genesis は PALW block ではなく、zero は default model を意味しない。

8.2 なぜ re-genesis なのか

Header v4/v5 は consensus preimage を変更するため、既存 chain に activation gate なしで途中導入できない。testnet-200 は DNS threshold を mid-chain 変更した結果、replay/IBD が poisoned invalid mark と missing-parent loop に入り得た。V4 は clean v5 re-genesis を選び、testnet-20 を public network とした。旧 state、UTXO、bond、PALW batch、MTPnetwork identity は持ち越さない。

8.3 v5 genesis identity

COMPUTE_REGISTRY_PALW_GENESIS は version 5、timestamp 2026-07-30 00:00 UTC、coinbase payload tag misaka-compute-registry、max-easy real-PoW fast-start bits 0x207fffff を持つ。新 genesis は model registry の schema を block 0 から固定し、最初の model registration のために再度 header fork を要求しない。

9 MODEL-AGNOSTIC COMPUTE SET REGISTRY

9.1 三層分離

Compute Set registry は、モデルに関する不変事実と、時間とともに変わる運用・経済設定を分離する。

Descriptor は write-once であり、一 bit の変更も新しい compute_set_id を作る。Policy は同一 set の lifecycle と economic parameters を sequence ごとに更新する。Allocation plan は全 active set を一括して再記述し、個別 patch を許さない。これにより、share 合計、zero-share、max active sets、rate limit を plan 全体で検証できる。

9.2 immutable descriptor

PalwComputeSetDescriptorV2 は次の root を持つ。

ID は

```
compute_set_id = Hash64k(misaka-palw-  
compute-set-id-v2, Borsh(D)).
```

ID は proposer が選ばず、canonical descriptor bytes から導出される。descriptor store は同一 ID・同一 bytes の再登録を idempotent とし、同一 ID・異なる bytes を local corruption または hashing fault として拒否する。

9.3 mutable policy

PalwComputeSetPolicyV1 は、少なくとも次を持つ。

- policy_sequence、effective_from_daa
- lifecycle state: Proposed、Shadow、Active、Deprecated、EmergencyHalted、Retired
- compute_work_scale、weight_factor_bps
- min_leaf_bond_sompi、job timeout、receipt retention
- auditor capacity threshold、premium π 、prompt/output token limits
- allowed shape set root

Policy ID は canonical policy bytes の content identity である。過去 block の評価は current policy ではなく、header が commit した exact policy record と source DAA を使う。Active policy は nonzero safety/economic fields を要求し、non-Active state は positive DAG weight を持てない。

9.4 atomic allocation plan

PalwModelAllocationPlanV1 は sequence、effective DAA、plan id、全 entry の compute_set_id と target_share_bps を持つ。entry は canonical order、unique set、bounded count であり、plan は active share を一括更新する。個別 set だけを patch すると、reorg や concurrent update で share sum が壊れるため、full replacement を採用する。

9.5 lifecycle

Retired は terminal。Deprecated/Halted からの再有効化は、より大きい sequence と future effective DAA を要求する。Emergency halt は policy revision より早く反映できるが、次の allocation plan で share を zero にする必要がある。

9.6 fork-local registry view

content store は descriptor/policy/plan/certificate を write-once で保持する。mutable state は block-keyed PalwComputeRegistryViewV1 に置き、

$$V_B = V_{SP(B)} \oplus \Delta_B$$

として accepted registry transactions を fold する。losing branch の content blob は残り得るが、view が選ばない限り governing state にはならない。reorg は block-keyed view の selected-parent chain を切り替えることで表現する。

10 REGISTRY TRANSACTIONS とモデル追加手順

10.1 固定 subnetwork band

V4 は registry transaction を一つの固定 band に置く。strict Borsh decode、trailing byte rejection、re-encode equality、version pin、unknown kind fail-closed を要求する。

10.2 新しいモデルを追加する実務手順

1. **Artifact freeze:** model weights、manifest、tokenizer、template、preprocessing、decode rule、integer arithmetic、LUT、shape/cost table、conformance vector を content-addressed package にする。

Table 3: Header version の evolution.

Version	追加 commitment	意味
v1	base header	pre-EVM / pre-PALW block format
v2	evm_commitment_root	selected-parent EVM commitment
v3	PALW ticket/work/beacon/auth fields	algo-4 audited-compute header
v4	spam accumulator commitment, spam nonce	public-network anti-spam re-genesis boundary
v5	set id, policy id, allocation plan id	model-agnostic registry reference

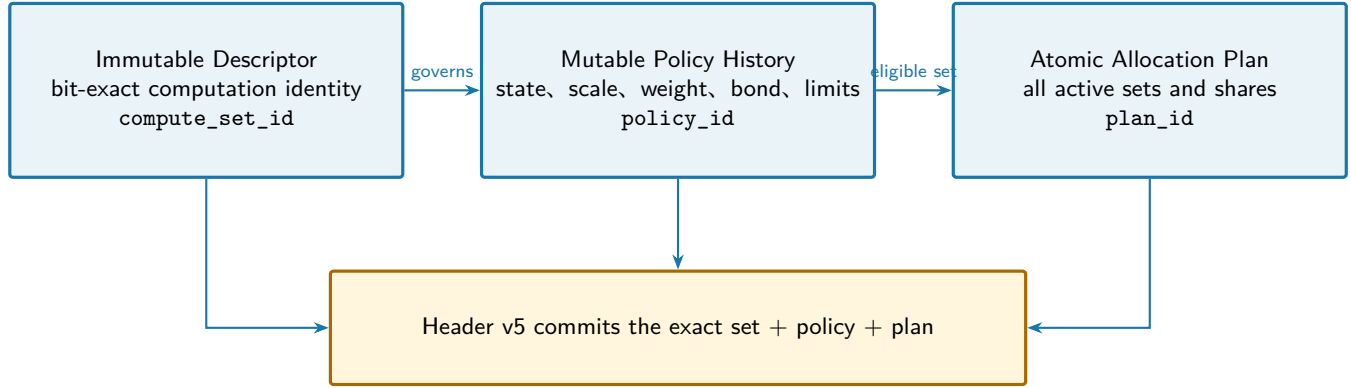


Figure 2: Compute Set の identity、economics、lane share を別 object として versioning する。

Table 4: Immutable Compute Set descriptor が束縛する field group.

Field group	束縛する内容
compute_vm_id	semantic program を解釈する固定 VM surface
model artifact / manifest	tensor bytes, topology, layer map, inventory
tokenizer / chat template / preprocessing	input bytes から token/tensor への canonical transformation
decode policy	greedy/top-k 等の decode rule, 停止条件, tie-break
semantic program	bounded Compute IR program root
shape / cost table	tensor shape と canonical compute-unit accounting
arithmetic / overflow / LUT	integer arithmetic, rounding, overflow budget, 近似 table
trace / checkpoint policy	replay checkpoint と dispute evidence の生成規則
conformance vector	implementation が満たす test vector set
modality / resource limits	text/vision/audio bitmask と static resource bound

Table 5: Compute Set lifecycle と historical treatment.

State	新規 activity	historical treatment
Proposed	job/ticket/mint 不可	descriptor と proposal history を保持
Shadow	execution/audit rehearsal, share 0	conformance と capacity evidence を収集
Active	job/ticket/block/credit 可	exact policy/plan で検証
Deprecated	新規 job 停止	既存 maturity/settlement と過去 block は有効
EmergencyHalted	新規 ticket/block を即時停止	history は無効化しない
Retired	terminal	record を保持し historical replay のみ

Table 6: Registry transaction の固定 subnetwork band.

Band	Payload	役割
0x40	Compute Set Proposal	descriptor, proposer credential, bond reference, artifact/reproduction root
0x41	Activation Certificate	DNS validator quorum による Shadow admission
0x42	Policy Update	lifecycle と economics の revision
0x43	Allocation Plan	active set 全体の share を atomic に更新
0x44	Emergency Halt	新規 ticket/block を即時停止

- Compute IR build:** semantic graph を supported Compute VMV1 へ lower し、acyclic reference, arity, instruction count, attribute bytes, tensor/resource bounds を静的検証する。
- Independent reproduction:** 少なくとも二つの独立 build path と二種類以上の backend class で descriptor root, conformance output, checkpoint root が一致する evidence を作る。
- Proposal:** 0x40 transaction で descriptor, artifact distribution root, independent build attestation root, requested Shadow DAA, proposal bond ref を登録する。
- Quorum certification:** DNS validators が artifact availability, conformance, auditor capacity, reproducibility evidence を確認し、ML-DSA-87 votes を含む 0x41 certificate を作る。
- Shadow policy:** 0x42 で Shadow state, share 0, work credit 0 の policy を future DAA から有効にする。
- Provider capability:** bonded provider は supported set root と capacity evidence を更新し、job assignment が未知 set に流れないようにする。
- Active policy:** soak, audit, DA, failure rate を満たした後、Active policy を future DAA で作る。work scale, weight, bond, timeout, premium は calibration evidence に基づく。
- Allocation plan:** 0x43 で全 active set の share を一括更新する。新 set だけでなく既存 set の share も plan に再記述する。

10. **Operation:** receipt/leaf が set id を commit し、mint path は leaf の set と fork-local governing policy/plan を header v5 に stamp する。
11. **Deprecation or halt:** 新規 job を止める場合は policy update、緊急停止は 0x44、退役は maturity/settlement 完了後の Retired policy を使う。

モデル追加時に node hard fork が不要となる条件

新 model が既存 compute_vm_id と opcode semantics、resource bound、wire format の範囲で表現できる場合、descriptor/policy/plan の登録だけで追加できる。新 opcode、arithmetic semantics、program encoding が必要なら新 Compute VM ID が必要であり、全 validating node がその VM を実装するまでは proposal を activate してはならない。model addition と VM addition は別の governance event である。

11 REGISTRY GOVERNANCE と残存 SECURITY GATE

11.1 canonical decode は authorization ではない

現在の fold path は accepted transaction を strict decode し、sequence/lifecycle/ramp rule を検証する。しかし payload が正しく decode できることは、その action を発行する権限を証明しない。proposal、policy update、allocation plan、emergency halt について、誰の署名、どの stake/bond quorum、どの timelock を要求するかを transaction validity へ接続する必要がある。

11.2 proposal bond

PalwComputeSetProposalV1 は proposal_bond_ref を持つが、現 snapshot では、その outpoint の value、owner、lock script、maturity、二重割当、rejected proposal burn、successful activation refund を registry admission が完全検証していない。field の存在は economic anti-spam の実装完了を意味しない。

11.3 header と governing view の一致

header-stage validation は committed descriptor、policy、plan を content store から解決し、resolve_source_policy_for_credit より ID equality、policy-set binding、effective DAA、Active state、nonzero share を確認する。このため、存在しない set、future-dated policy、inactive policy、zero-share plan は reject される。

しかし現 validator は、header が commit した policy/plan が「その DAA で有効な record」であることは確認するが、**fork-local registry view が同 DAA に選ぶ highest-sequence governing policy/plan と同一であることを確認しない**。mint path は正しい governing refs を選ぶが、malicious producer が古い Active policy と古い effective plan を commit して self-consistent に見せる余地がある。

Required equality check

Header/body validation は selected-parent から導出される chain-stage-bound registry view V_B を取得し、

$$(p^*, s^*, a^*) = V_B.\text{governing_references_at}(\text{set}, \text{DAA}(B))$$

を再計算した上で、header の policy_id = p^* 、plan_id = a^* 、 $s^* = \text{Active}$ を要求する。content-only resolver と fork-local governing resolver の両方が必要である。

11.4 activation certificate identity

activation certificate store は現在 compute_set_id を key にするため、同じ set の re-certification、expiry、supersession を write-once content store で表現できない。certificate 自身の content hash を

key にし、view が current/superseding certificate id を選ぶ構造へ移す必要がある。

11.5 fold failure posture

registry fold は semantic rejection を warn + skip とする。transaction validity が payload shape だけを保証する現状では、unauthorized または invalid action が block acceptance と registry effect の不一致を生む。public-value activation では、consensus-critical governance admission を transaction contextual validity へ移し、accepted transaction が deterministic effect を持つことを保証する必要がある。

12 COMPUTE IR V1 と RUNTIME PLUGIN

12.1 目的

モデルごとに Rust native verifier を node binary へ追加すると、model upgrade が consensus software upgrade になり、attack surface と review cost が増える。Compute IR は、descriptor が参照する semantic program を bounded integer VM surface へ落とし、runtime implementation を external plugin として交換可能にする。

12.2 frozen VM identity

Compute VM V1 は arithmetic tag int-deterministic/rne/overflow-budgeted/v1、encoding tag borsh-le/explicit-discriminants/v1、31 opcode の code/name/arity table を canonical surface string にし、

$$\text{compute_vm_id} = \text{Hash64}_k(\text{compute_vm_id}, \text{VM surface})$$

で固定する。opcode の renumbering は禁止し、追加は新 VM version と新 ID を要求する。

12.3 opcode classes

12.4 static validation

Program は version と VM ID を確認し、empty program、unknown opcode、forward/self reference、arity violation、instruction count、attribute bytes、tensor count、resource limit を fail-closed で拒否する。value reference は input tensor または strictly earlier instruction output のみを指し、graph は構造上 acyclic になる。

12.5 runtime plugin boundary

palw-runtime-plugin/src/lib.rs は external runtime/replay interface を提供する。node は descriptor/VM/program identity と proof/evidence を検証し、model-specific execution engine は plugin 側へ分離する。Metal と CUDA は PALW runtime acceleration の optional backend であり、node、misaminer、kasper-pq-miner の block validation/mining は CPU-only である。

12.6 現状の不足

Compute IR type と static validator は実装済みだが、proposal admission が semantic program payload を DA から fetch し、root、VM ID、resource limits、program validation を一体で検証する path は未完成である。descriptor の root field だけを信頼して activation してはならない。

13 PALW OVERVIEW と THREAT MODEL

13.1 PALW の対象

PALW は Proof of Audited LLM Work の略であり、登録済み deterministic computation の execution claim を、commit、assignment、replication、DA、audit、maturity、ticket、block authorization へ変換する。full node は block path でモデル全体を再実行せず、commitment と certificate chain を検証する。

Table 7: Compute VM V1 の opcode class.

Class	Representative opcodes	semantics
loading / unpack	Int4Unpack, Int8Load	canonical weight/activation byte interpretation
linear algebra	Int8Gemm, Int32Reduce, Int64Requantize	bounded accumulation, requantization, overflow budget
integer primitive	RneShift, Clamp, Add, Subtract, Multiply, DivideFloor, IntegerSqrt	fixed rounding と integer-only transformation
index / order	Gather, Scatter, TopK, StableSort	deterministic tie-break と canonical ordering
nonlinear	LookupTable, RmsNormFixed, SoftmaxFixed, RoPeFixed, ActivationFixed	LUT または fixed-point approximation
state / KV	StateRead, StateWrite, KvRead, KvWrite	recurrent/KV state transition
vision / MoE	ConvInteger, PatchEmbeddingInteger, ExpertDispatch, ExpertMerge	bounded multimodal と expert routing
checkpoint	MerkleCheckpoint	replay segment commitment

13.2 adversary capability

adversary は provider/auditor identity を多数作り、commit 後に data を withholding し、requester と collude し、stale model/policy を選び、同じ work を複数回 claim し、DA peer を flood し、header field を transplant しようとする。また、runtime difference、overflow、tie-break、tokenizer drift により honest backend 間 divergence が起こり得る。

13.3 security assumptions

安全性は、ML-DSA-87、BLAKE2b-512、stake/bond economic cost、future beacon unpredictability、canonical integer semantics、public data availability、少なくとも一部 honest auditor、permanent hash floor に依存する。モデル出力の semantic truth は assumption に含めない。

14 PROVIDER REGISTRY, SELF-ORDER JOB, PCPB

14.1 single bonded provider registry

V3 から、hardware vendor などの pool を廃止し、一つの bonded registry から primary worker、replica、auditor を選ぶ。provider record は owner key、bond outpoint、supported Compute Set root、capacity、session authorization を持つ。vendor/device metadata は telemetry であり、consensus identity ではない。

14.2 self-order job

requester 自身が primary result を生成する self-order job では、prompt/output を後から都合よく選び直す攻撃が強い。job-specific challenge を input context に埋め、primary は result の hiding commitment と escrow を先に固定する。future beacon 後に replica を選び、primary result を見せずに独立 regeneration を行う。

14.3 PCPB flow

PCPB は pre-commitment provider binding を意味する。

1. requester/primary が job、challenge、input commitment、result commitment、escrow を commit する。
2. future beacon で replica provider を bonded capacity から選ぶ。
3. replica は primary reveal 前に independent execution commitment を提出する。
4. primary が reveal し、exact result/checkpoint match を判定する。
5. DA window と audit maturity を通過した leaf だけが ticket になる。

現 snapshot では PCPB security model、fraud carrier、slash semantics は ADR に定義されるが、public-value path への wiring は fenced されている。claim-on-fetch を完全に置換するには、requester/provider assignment authenticity、withholding timeout、escrow release、objective fault evidence を同じ state machine に接続する必要がある。

15 RECEIPT, CHECKPOINT, DA, LEAF

15.1 receipt identity

Compute receipt は network/job/batch、provider credential、compute set id、input root、challenge、output root、route/state root、checkpoint execution root、compute units、timing、implementation telemetry を commit する。V4 では set id が model/runtime identity の中心であり、vendor runtime class は consensus match key ではない。

15.2 checkpoint execution root

全 instruction の op-level trace を通常時に公開すると、hash tax と DA cost が大きすぎる。通常 path は interval ごとの checkpoint state を Merkle chain にし、紛争時だけ divergent segment を op-level trace へ展開する。

$$r_{i+1} = \text{Hash}_{64k}(\text{checkpoint}, r_i \parallel i \parallel \text{state}_i \parallel io_i).$$

final root は output token だけでなく、expert route、recurrent/KV state、semantic schedule、canonical compute units を束縛する。

15.3 DA Object V2

DA object は manifest、chunk root、erasure/reconstruction meta-data、owner key、session authorization、challenge/response、retention、slash state を持つ。permissionless peer は manifest/chunk proof を取得でき、owner だけに依存しない fetch path を必要とする。automatic owner-key response、multi-peer long soak、incident runbook は運用 gate として残る。

15.4 leaf と exact match

primary/replica の exact match、bond、assignment、DA availability、receipt membership を満たすと leaf が形成される。leaf は receipt_v3_compute_set_id を commit し、後の mint path で set identity の唯一の入力になる。miner が好きな model を選ぶのではない。

15.5 nullifier

work claim は leaf coordinate から導出した one-shot nullifier を持つ。block acceptance、reward application、pruning snapshot、replay reconstruction の全 path で同じ nullifier identity を使う。full-lifecycle prune/replay E2E は実装済みだが、PCPB 本体と fraud evidence の link は別 gate である。

16 OBJECTIVE AUDIT と CERTIFICATES

16.1 three-layer audit

1. **Structural**: signature、bond、assignment、timing、Merkle membership、canonical encoding。
2. **Exact semantic**: output、route、state、checkpoint root、compute units の一致。
3. **Sample full replay**: future beacon で選ばれた leaf/segment を auditor が full replay する。

16.2 batch certificate

既存 PALW batch certificate は selected auditor slate と sample plan を beacon から再導出し、missing vote を quorum denominator に残す。auditor が abstain して denominator を小さくすることを許さない。certificate store は content hash で key され、fork-local lifecycle view が exact certificate を選ぶ。

16.3 set-centric audit certificate

PalwComputeSetAuditCertificateV1 は generic Compute Set architecture のための型と content hash を持つ。しかし現 snapshot では、embedded auditor votes、evidence root、plugin replay result、boolean pass を end-to-end consensus verifier へ接続していない。API の bool field だけでは客観 replay proof にならない。

16.4 dispute と TraceVM

correctness slashing は、divergent integer step を consensus-verifiable TraceVM が裁定できるまで無効にする。先に自動化できるのは、署名不正、double commit、invalid opening、deadline miss、DA refusal、unauthorized spend のような objective fault である。semantic mismatch を単純 majority vote で slash すると、honest minority runtime を破壊する。

17 TICKET CREATION と ALGO-4 BLOCK

17.1 ticket eligibility

leaf は certificate、DA、audit、maturity、unused nullifier、active set、nonzero share を満たして ticket になる。ticket は compute output の価値ではなく、protocol history を通過した one-shot work claim である。

17.2 block authorization

PALW block producer は ML-DSA-87 authorization で header preimage 全体を束縛する。ticket ref、nullifier、chain commit、beacon seed、component work、anti-spam commitment、set/policy/plan reference の transplant を防ぐため、authorization digest は block identity と同じ coordinate を覆う必要がある。

17.3 construction order

1. DNS-confirmed anchor と beacon seed を解決する。
2. on-chain leaf と certificate を store から取得する。
3. registry-active net では leaf の set id と governing policy/plan を sink view から取得する。
4. per-set share から lane bits を構築し、ticket/nullifier/chain commit を header に置く。
5. block body、coinbase/reward state、anti-spam accumulator を確定する。
6. final header preimage に producer authorization を作る。
construction と validation は同じ helper と coordinate を使い、node が自分で作った block を自分で reject する分岐を避ける。

17.4 header v5 references

PALW lane の v5 header は三 field を持つ。

- palw_compute_set_id
- palw_compute_policy_id
- palw_allocation_plan_id

Hash lane の v5 header は zero reference を持つ。PALW lane では unknown descriptor を reject し、policy/plan record が local store に無い場合は trusted-data availability fault として fail-stop する。

18 PER-SET DIFFICULTY と HISTORICAL CREDIT

18.1 sublane

registry-active network では、PALW replica lane を Compute Set ごとの logical sublane として扱う。allocation plan の target_share_bps が difficulty retarget の target interval を決める。share 0 は block eligibility 0 であり、minimum default share へ fallback しない。

18.2 mint path

標準 mint path は leaf の receipt_v3_compute_set_id を読み、sink block の fork-local registry view から target DAA における governing_references_at を呼ぶ。state が Active、plan share が nonzero であることを確認し、その exact references を template header に stamp する。

18.3 source-policy resolution

GHOSTDAG credit は current policy を参照せず、source header が commit した exact policy/plan と source DAA を使う。policy update 後に過去 block の work が増減しないことが重要である。missing historical record を zero/default へ落とすと fork-choice divergence を起こすため、availability fault は fail-stop とする。

18.4 retention requirement

pruning snapshot は、unpruned header が参照する全 descriptor、policy、plan、certificate、fork-local view proof を含める必要がある。content store の record は chain length ではなく governance cadence で増えるため、aggressive pruning より complete historical resolution を優先する。

19 HEADER-V4 ANTI-SPAM と RESOURCE SAFETY

19.1 anti-spam accumulator

Header v4 は authenticated spam accumulator commitment と independent spam nonce を preimage に入れる。algo-4 block が real hash floor exemption を持つ構造では、ticket verification 以前に cheap header flood を受けやすいため、non-inert spam work を要求する。

19.2 sibling-flood bounding

GHOSTDAG reachability/allocator は valid sibling flood に対する structural bound を持つ。単一 peer が多数 sibling を送り、allocation/reachability memory を無制限に増やすことを防ぐ。multi-machine long soak と independent review は public-value gate として残る。

19.3 caps

Compute Set view cap 256、activation certificate vote cap 4096、program/resource bound、active set cap、plan ramp limit は structural anti-explosion control である。economic bond は structural cap を代替せず、両方が必要である。

20 SELECTED-PARENT EVM

20.1 architecture

EVM lane は selected-parent execution に従い、header v2 の evm_commitment_root で state/receipt/bridge outcome を block identity に束縛する。DAG の全 sibling を EVM 順序へ入れるのではなく、selected-parent chain に deterministic execution order を与える。

20.2 bridge accounting

UTXO deposit は consume/credit、withdraw は EVM debit/UTXO materialization を行い、同一 deposit の二重 claim、unauthorized bridge spend、commitment mismatch を拒否する必要がある。repository には execution、RPC、bridge tests が存在する。

20.3 PQ scope

base ledger は PQ-only だが、Ethereum typed transaction compatibility path は ECDSA sender recovery を含む。PREA/PQ account design は ML-DSA verification handler、registered account、session key、strict deposit lock を検討するが、これをもって EVM 全体が PQ-only になったとは言えない。

20.4 現 status

EVM crates、feature flag、execution、RPC、bridge tests は repository に存在する。しかし testnet-20 は `evm_activation_daa_score=u64::MAX` を継承し、EVM lane は active ではない。V4 title から Native EVM の public-service claim を外し、architecture component として記述する。

21 REWARD、TOKENOMICS、MTP

21.1 separation of acceptance and payment

PALW algo-4 acceptance、positive DAG work、provider payment は別 lever である。testnet-20 は algo-4 と registry path を開くが、`flat_palw_compute_work_scale` は zero。個別 policy の nonzero weight を public-value network へ持ち込む前に、replay、audit、bond、governance、calibration を閉じる必要がある。

21.2 theoretical supply

code comment と subsidy table は、10B MSKgenesis premine と約 16.013224875B の 30 年 emission、年率 1.4% decay $q = 0.986$ を示す。theoretical max は約 26.013224875B MSK である。

$$S_{\text{theory}} = 10\text{B} + \sum_{m=0}^{359} R_0 q^{m/12} \approx 26.013224875\text{B}.$$

per-block reward を integer sompi へ切り上げるため、10 BPS の live cumulative issuance は theoretical figure より約 44 MSK 上回り得る。MAX_SOMPI は per-amount sanity cap であり、厳密な cumulative emission cutoff ではない。

21.3 provider economics

policy は leaf bond floor、premium π 、timeout、retention、auditor threshold、work scale/weight を持つ。これらを変更できる actor が未認可なら、任意の node/transaction sender が経済設定を操作できる危険がある。枚数を預けるだけではなく、bond owner、governance role、quorum、timelock、slash/refund condition を一体で定義する必要がある。

21.4 MTP

MTP は testnet participation record であり、token、on-chain balance、tradable asset、future distribution entitlement ではない。network re-genesis を跨いで自動移行しない。point calculation と将来の tokenomics decision は分離し、現時点の point が将来の配布枚数を保証しない。

22 SECURITY ANALYSIS

22.1 threat matrix

Table 8: Security threat matrix。 (1/2)

脅威	現在の防御	残余
classical signature break unknown model/VM fallback	ML-DSA-87 transaction auth、classical path exclusion content-derived set id、explicit VM id、unknown discriminant fail closed	transport PQ、implementation side-channel review proposal admission が IR payload を検証していない
historical policy rewrite registry state spoof proposal spam activation by one key auditor withholding	header commits exact policy/plan、source-DAA resolver fork-local selected-parent view、write-once content store tx cost、max 256 sets、bond ref field DNS validator ML-DSA-87 stake quorum selected slate denominator、missing vote counts against quorum	highest-sequence governing view equality が未検証 policy/plan/halt governance authorization が無い bond value/owner/lock/maturity/burn/refund 未検証 testnet minimum 1 validator、certificate re-key issue set-centric audit verifier と public soak

Table 9: Security threat matrix。 (2/2)

脅威	現在の防御	残余
DA withholding	chunk proof、challenge/response、timeout、slash state、multi-peer fetch	automatic owner-key response、long soak、incident operations
duplicate work claim provider collusion algo-4 spam valid sibling flood unsafe parameter change EVM bridge inflation	leaf-bound nullifier、reward-coordinate dedup、prune/replay E2E future beacon、independent replica design、PCPB target Header-v4 accumulator、spam nonce、real hash lane bounded reachability allocator versioned fields、params identity、re-genesis migration deposit consume/credit、withdraw debit/materialize、UTXO commitment	provider assignment authenticity と fraud link PCPB/fraud carrier/slash wiring inactive economic calibration と distributed flood campaign multi-machine long soak、independent review governance process と release discipline EVM inactive、PQ account/strict lock rollout

22.2 model correctness と truth

Exact replay は、同じ descriptor、input、challenge、integer semantics の下で同じ commitment が得られたことを示す。これは semantic truth、factuality、alignment、copyright compliance、safety を判定しない。PALW は computation claim の一部を検証する protocol であり、answer truth oracle ではない。

22.3 hardware attestation を identity にしない理由

NVIDIA、AMD、Apple の runtime telemetry や driver identity は consensus key ではない。hardware-specific floating-point behavior、kernel fusion、library version、non-deterministic scheduling は cross-vendor exact match を壊す。V4 は canonical integer VM と artifact roots を identity にし、hardware は performance telemetry と deployment metadata に留める。

22.4 fail-stop と liveness

missing historical registry record は default に落とさず local fail-stop とする。これは safety を優先するが、trusted-data package、snapshot completeness、record retention が不十分なら IBD liveness を失う。registry-active pruning specification は availability set を明示し、snapshot builder/verifier を同じ root へ束縛する必要がある。

23 IMPLEMENTATION STATUS FROM CODE

23.1 status matrix

Table 10: Code snapshot から確認した implementation status。(1/3)

項目	状態	Code evidence	判定
PQ-only ML-DSA-87 base	実装済み	crypto/txscript, consensus/core/src/config/params.rs, ADR-0019	testnet active
Hash64 identity	実装済み	consensus/core/src/hashing, ADR-0008	testnet active
DNS two-dimensional finality	実装済み	consensus/core/src/dns_finality.rs, params	testnet active
Header v5 fields/preimage	実装済み	consensus/core/src/header.rs, hashing/header.rs	testnet-20 active
Compute Set descriptor/policy/plan	実装済み	consensus/core/src/palv_compute_set.rs	types/rules/tests
content-addressed record stores	実装済み	model/stores/palv_compute_registry.rs	write-once tiers
fork-local registry view	実装済み	virtual_processor/processor.rs	block-keyed fold
activation quorum verifier	実装済み	processes/palv.rs	embedded ML-DSA votes
per-set difficulty	実装済み	header_processor/pre_pow_validation.rs, processes/difficulty.rs	live code path
historical credit resolver	実装済み	palv_compute_set.rs, GHOSTDAG seam	exact record input

Table 11: Code snapshot から確認した implementation status。(2/3)

項目	状態	Code evidence	判定
Compute IR V1	部分実装	consensus/core/src/palv_compute_ir.rs	31 opcodes/static validation
runtime plugin	部分実装	palv-runtime-plugin/src/lib.rs	interface, external replay pending
proposal governance auth	未解決	registry tx decode/fold	no signer/quorum admission
policy/plan/halt auth	未解決	registry tx decode/fold	economics mutable by accepted payload
proposal bond lock	未解決	proposal has outpoint field	value/owner/lock absent
bond maturity/burn/refund	設計のみ	ADR/governance design	no lifecycle wire
header governing-view equality	未解決	content resolver only	stale active revision possible
certificate content-key	未解決	store keyed by set id	re-certification conflicts
IR payload admission	ゲート中	descriptor has roots	DA fetch/root/VM validation missing

Table 12: Code snapshot から確認した implementation status。(3/3)

項目	状態	Code evidence	判定
set-centric audit replay	ゲート中	audit cert type/hash	vote/replay verifier missing
PCPB/fraud/slash wiring	ゲート中	ADR-0045/security model	public path fenced
DA Object V2	部分実装	consensus/core/src/palv_da.rs, DA processes	external soak remains
permissionless snapshot auth	実装済み	ADR-0042, pruned frontier	fence completed
nullifier prune/replay E2E	実装済み	ADR-0044, tests	full lifecycle test
sibling-flood bound	実装済み	ADR-0043, reachability allocator	external soak remains
Header-v4 anti-spam	実装済み	palv_antispam.rs, v4/v5 genesis	testnet active
selected-parent EVM	ゲート中	kaapa-evm, EVM processes	testnet-20 inactive
mainnet	ゲート中	params/constants defined	not launched

23.2 documentation drift

README, params, migration note は testnet-20 を public network とする一方、一部 source comment と participation guide は「no shipped preset activates v5」または testnet-200 を残す。operator-facing documentation, CLI default, seed domain, port, MTP rule を params identity から自動生成し、stale prose を release gate で検出する必要がある。

24 TESTNET LADDER と ACTIVATION GATES

24.1 ladder

24.2 StopShip conditions

次のいずれかが残る状態で positive-value activation を行ってはならない。

- unauthorized actor が policy, allocation, halt を適用できる。
- proposal bond reference が実 UTXO lock と一致しない。
- header が stale policy/plan を fork-local governing view より優先できる。
- re-certification が store key collision で失われる。
- unsupported VM, missing IR payload, invalid resource limit が activation を通る。
- auditor PASS が replay evidence と署名へ束縛されない。
- nullifier, reward, slash, pruning snapshot の coordinate が一致しない。
- public peer が DA object を owner 依存なしで取得・検証できない。
- economic parameter が distributed measurement と incident rollback なしで設定される。

25 GOVERNANCE, VERSIONING, MIGRATION

25.1 governance object

将来の governance action は、actor credential, action type, payload hash, network id, sequence, effective DAA, expiry, bond/quorum proof, timelock を署名 summary に含めるべきである。proposal, policy, plan, halt は異なる権限と緊急性を持つため、一つの万能管理鍵に集約しない。

25.2 parameter classes

25.3 migration rule

network identity, genesis, header preimage, activation semantics を変える場合、old state を暗黙変換しない。migration note は、new network id, ports, genesis hash, params identity, fresh datadir, seed domain, wallet/address prefix, bond re-registration, MTPscope を一つの manifest として署名・公開する必要がある。

25.4 model and VM versioning

model weights, tokenizer, template, decode rule, arithmetic, LUT, program root の変更は new set ID。economics だけの変更は new policy。share 変更は new allocation plan。new opcode/semantics は new VM ID であり、software rollout と activation quorum を先に行う。unknown VM を approximate または generic fallback で実行してはならない。

26 LIMITATIONS と OPEN QUESTIONS

- Governance root:** DNS validator quorum を registry governance に使うか、別 governance bond set を持つか。
- Proposal economics:** bond floor, maturity, active proposal cap, rejected proposal burn, successful activation refund, spam slash をどう定義するか。

Table 13: Testnet ladder と exit criteria。

Stage	Gate	Exit criteria
S0 Structural	code landed	canonical encoding、version gates、unit/property tests、unknown input fail closed
S1 Closed replay	two controlled hosts	same descriptor/IR/conformance roots、reorg、restart、IBD、prune/replay
S2 Shadow model	share 0 / weight 0	independent artifact build、Mac/RTX or cross-vendor exact match、DA availability
S3 Public no-value	testnet-20	permissionless peers、auditor churn、withholding、flood、recovery runbook
S4 Economic rehearsal	bounded test parameters	governance auth、bond lock、maturity/burn/refund、calibrated spam/work scales
S5 Positive compute credit	nonzero policy weight	historical resolver + governing-view equality、external review、long soak
S6 Provider payment/slash	money path	objective fault verifier、PCPB、escrow、fraud carrier、appeal/incident process
Mainnet candidate	immutable ceremony	independent audit、signed manifest、fresh genesis、reproducible builds、launch governance

Table 14: Parameter class と change rule。

Class	Examples	Change rule
consensus identity	header version、genesis、hash domain、VM surface	re-genesis or explicit hard fork
immutable set identity	artifact/tokenizer/program/arithmetic roots	new Compute Set ID
mutable set policy	state、work scale、weight、bond、timeouts、premium	authorized future-Daa policy revision
allocation	active set shares	authorized atomic full plan
emergency	halt evidence root	fast quorum/timelock path、history preserved
telemetry	GPU model、driver、throughput	no consensus effect
measurement evidence	soak result、failure rate、calibration	signed/public artifact、policy 変更の根拠

3. **Certificate multiplicity**: re-certification、expiry、supersession、fork-local latest certificate をどう index するか。
4. **Compute IR DA**: semantic program、shape table、LUT、artifact manifest をどの DA namespace と retention rule で供給するか。
5. **Audit verdict**: boolean pass をどの evidence root と signed vote に束縛し、partial replay と full replay をどう区別するか。
6. **PCPB confidentiality**: provider assignment 前の output leakage、requester/provider collusion、withholding をどの commitment と timeout で裁定するか。
7. **Economic calibration**: spam cost、compute-to-hash scale、premium、slash fraction、bond floor を multi-machine campaign で測定する必要がある。
8. **Model genesis**: official weight provenance、license、two independent builds、Mac/RTX cross-vendor exact reproduction が外部依存である。
9. **EVM PQ scope**: Ethereum compatibility と PQ-only account authorization を同時に提供する strict deposit、account factory、session key、RPC UX が未完である。
10. **Documentation drift**: public operator guide、CLI default、MTP rule、network ports を source params と同時更新する release automation が必要である。

27 結論

MISAKA Version 4 は、検証可能 LLM work を「特定モデルの特別扱い」から「登録された bit-exact computation の versioned state machine」へ移す。Header v5 は set、policy、allocation plan を block identity に束縛し、Compute Set registry は immutable identity と mutable economics を分離する。Compute IR V1 はモデル別 native code を避け、per-set difficulty と historical record resolution は将来の share/policy 変更から過去 work を切り離す。

提出 snapshot は、PQ base、DNS finality、public testnet-20、v5 header、registry stores/views、activation quorum、algo-4、anti-spam、DA/snapshot/pruning の大きな縦断面を実装している。同時に、governance authorization、proposal bond、governing-view equality、certificate identity、IR admission、set-centric audit replay、PCPB/fraud、external reproduction は未解決である。

したがって V4 の正しい読み方は「モデル追加可能な architecture が code に入った」であり、「誰でも安全にモデルを追加でき、経済価値を持つ mainnet が完成した」ではない。次の milestone は機能を増やすのではなく、authority、bond、historical view、

audit evidence、data availability を同じ consensus coordinate で閉じ、no-value testnet で壊すことである。

A CONSENSUS OBJECT SKETCH

A.1 descriptor、policy、plan

```
PalwComputeSetDescriptorV2 {  
  version,  
  compute_vm_id,  
  model_family_id,  
  model_artifact_root,  
  model_manifest_root,  
  tokenizer_root,  
  chat_template_root,  
  preprocessing_root,  
  decode_policy_root,  
  semantic_program_root,  
  shape_table_root,  
  shape_cost_table_root,  
  arithmetic_rules_root,  
  overflow_budget_root,  
  lut_root,  
  trace_policy_root,  
  checkpoint_policy_root,  
  conformance_vector_root,  
  modality_mask,  
  resource_limits_root  
}
```

```
PalwComputeSetPolicyV1 {  
  compute_set_id,  
  policy_sequence,  
  effective_from_daa,  
  state,  
  compute_work_scale,  
  weight_factor_bps,  
  min_leaf_bond_sompi,  
  job_timeout_daa,  
  receipt_retention_daa,  
  auditor_capacity_threshold,  
  premium_pi_bps,  
  max_prompt_tokens,  
  max_output_tokens,  
  allowed_shape_set_root  
}
```

```
PalwModelAllocationPlanV1 {  
  sequence,  
  effective_from_daa,  
  entries[] = { compute_set_id, target_share_bps },  
  plan_id  
}
```

A.2 activation certificate

```
PalwComputeSetActivationCertificateV1 {  
  compute_set_id,  
  descriptor_hash,  
  conformance_result_root,  
  auditor_capacity_evidence_root,  
  artifact_reproducibility_root,  
  validator_set_commitment,  
}
```

```

    approving_stake,
    total_selected_stake,
    effective_from_daa,
    votes_root,
    votes[] = { bond_outpoint, vote, mlds87_signature }
}

```

A.3 header v5 extension

```

HeaderV5Extension {
    palv_compute_set_id,
    palv_compute_policy_id,
    palv_allocation_plan_id
}

```

B VERIFIER PSEUDOCODE

B.1 descriptor admission

```

function admit_descriptor(proposal, tx_context):
    require strict_borsh(proposal)
    require proposal.descriptor.version == 2
    require proposal.descriptor.modality_mask != 0
    set_id = Hash64(domain_set_id, borsh(proposal.descriptor))

    ir_bytes = DA.fetch(proposal.descriptor.semantic_program_root)
    require Hash64(domain_ir, ir_bytes) == descriptor.semantic_program_root
    ir = strict_borsh(ir_bytes)
    require ir.compute_vm_id == descriptor.compute_vm_id
    require node.supports(ir.compute_vm_id)
    require validate_compute_ir_program(ir, descriptor.resource_limits_root)

    bond = UTXO.lookup(proposal.proposal_bond_ref)
    require bond.owner == authorized_proposer_or_governance_script
    require bond.value >= MIN_PROPOSAL_BOND
    require bond.script == proposal_bond_lock_script(set_id, maturity)
    require proposal.proposal_bond_ref not already assigned

    require governance_authorized(PROPOSAL, Hash64(proposal), tx_context)
    insert_write_once(set_id, proposal.descriptor)

```

B.2 activation certificate

```

function verify_activation(cert, selected_parent_bond_view, network_id, daa):
    require cert.votes is sorted and unique by bond_outpoint
    require len(cert.votes) <= 4096
    require cert.validator_set_commitment == commitment(selected_parent_bond_view)
    require cert.votes_root == merkle_or_hash(cert.votes)

    approving = 0
    total = total_active_stake(selected_parent_bond_view)
    for vote in cert.votes:
        bond = selected_parent_bond_view[vote.bond_outpoint]
        require bond.active_at(daa)
        require MLDS87.verify(bond.validator_key, vote.signing_hash(network_id, cert), vote.
            ↳ signature)
        if vote.vote == PASS: approving += bond.amount

    require approving == cert.approving_stake
    require total == cert.total_selected_stake
    require 3 * approving >= 2 * total
    return true

```

B.3 header governing-reference validation

```

function validate_v5_palv_header(B):
    require descriptor_store.contains(B.compute_set_id)
    policy = policy_store[B.compute_policy_id]
    plan = plan_store[B.allocation_plan_id]
    require resolve_source_policy_for_credit(policy, plan, B.daa) succeeds

    V = registry_view_at(selected_parent(B))
    (governing_policy, state, governing_plan) =
        V.governing_references_at(B.compute_set_id, B.daa)
    require state == Active
    require B.compute_policy_id == governing_policy
    require B.allocation_plan_id == governing_plan
    require share(governing_plan, B.compute_set_id) > 0

```

B.4 proposal bond lifecycle target

```

function settle_proposal_bond(proposal, outcome, current_daa):
    o = UTXO.lookup(proposal.proposal_bond_ref)
    require o.script == proposal_bond_lock_script(
        proposal.compute_set_id,
        maturity_daa,
        refund_path,
        burn_or_slash_path)

```

```

require o.value >= MIN_PROPOSAL_BOND
require current_daa >= o.creation_daa + BOND_MATURITY
require proposal.proposal_bond_ref not assigned to another proposal

if outcome == ACTIVATED_AND_REPRODUCIBLE:
    authorize(refund_path)
else if outcome in {SPAM, FRAUD, DA_REFUSAL, EXPIRED}:
    authorize(burn_or_slash_path)

```

C CODE TRACEABILITY

Table 15: Code traceability matrix。 (1/3)

Code / document	本書での役割
README.md	public testnet status、hardware、tokenomics、claim boundary
consensus/core/src/constants.rs	Header v2-v5 version boundaries
consensus/core/src/config/params.rs	testnet-20/testnet-200/mainnet activation、DNS、PALW、EVM params
consensus/core/src/config/genesis.rs	v4/v5 genesis identity
consensus/core/src/palw_compute_set.rs	descriptor、policy、plan、registry effects、view、capability、audit cert
consensus/core/src/palw_compute_ir.rs	Compute VM V1、opcode、static validation
palw-runtime-plugin/src/lib.rs	external runtime/replay interface
consensus/src/model/stores/palw_compute_registry.rs	content stores、certificate key、block-keyed view
consensus/src/pipeline/virtual_processor/processor.rs	registry fold、activation certificate verification、view commit

Table 16: Code traceability matrix。 (2/3)

Code / document	本書での役割
consensus/src/pipeline/header_processor/pre_pow_validation.rs	v5 set/policy/plan resolution、per-set sublane
consensus/src/consensus/palw_mint.rs	leaf-derived set、governing refs、algo-4 template
consensus/src/processes/palw.rs	certificate verification、PALW binding、tests
consensus/core/src/palw.rs	receipt/leaf/batch certificate/nullifier/core parameters
consensus/core/src/palw_pruned_frontier.rs	pruned IBD snapshot authentication
consensus/core/src/palw_antispam.rs	Header-v4 anti-spam
docs/testnet-20-migration.md	public network cutover、unsafe replay rationale
docs/adr/0040-palw-single-pool-integer-canonical-remediation.md	PALW security gates、status ledger
docs/adr/0042-permissionless-snapshot-auth-completion.md	snapshot fence completion

Table 17: Code traceability matrix。 (3/3)

Code / document	本書での役割
docs/adr/0043-g6-sibling-flood-bounding.md	reachability resource bound
docs/adr/0044-nullifier-prune-replay-e2e.md	full lifecycle pruning test
docs/adr/0045-pcpb-fraud-audit-security-model.md	PCPB/fraud semantics
docs/adr/0046-economic-parameter-calibration.md	economic calibration
docs/adr/0047-model-genesis-dual-reproduction.md	external model reproduction
docs/adr/0048-header-v4-staging-mainnet.md	anti-spam re-genesis staging
kaspa-evm、consensus/src/processes/evm	selected-parent EVM/bridge implementation

D PARAMETER SUMMARY

Table 18: testnet-20 と implementation parameter の summary。 (1/2)

Parameter	testnet-20 / implementation value	備考
network id	testnet-20	public testnet
header version	5	Compute Set references active
P2P port	26521	public seed network
PQ activation DAA	0	genesis-active
DNS activation DAA	0	genesis-active
PALW activation DAA	0	staging shape inherited
Compute registry activation DAA	0	open from genesis
algo-4 acceptance	true	acceptance does not imply weight/payment
flat PALW compute work scale	0	per-set policy remains governing for registry path
EVM activation DAA	u64::MAX	inactive
DNS min bond	10 MSK	testnet entry floor
DNS min active stake	10 MSK	testnet entry floor

Table 19: testnet-20 と implementation parameter の summary。 (2/2)

Parameter	testnet-20 / implementation value	備考
DNS min active validators	1	testnet bring-up
required work depth	100 work units	two-dimensional confirmation
required stake depth	5000 scaled stake units	two-dimensional confirmation
finality depth	432,000 blocks	full-scale inherited
pruning depth	1,080,000 blocks	full-scale inherited
PALW active batch window	16 epochs	registration/audit overlap
Compute Set view cap	256 sets	structural anti-explosion cap
activation cert vote cap	4,096	structural cap
provider bond floor	10 MSKprovisional	ADR-0046
annual emission decay	1.4% ($q = 0.986$)	30-year table
theoretical max supply	about 26.013224875B MSK	10B premine + emission

E VERSION 3 から VERSION 4 への対応

Table 20: Version 3 concept と Version 4 representation の対応。

V3 concept	V4 representation	変更理由
single canonical compute set	multiple immutable descriptors	model addition without node hard-code
QW36 runtime class	descriptor + Compute VM ID + artifact roots	telemetry と consensus identity の分離
single pool allocation	atomic allocation plan	multi-set share の整合的更新
fixed economic parameters	source-DAA policy history	historical reproducibility
hardware pool name	provider capability supported-set root	vendor independence
header v3 PALW fields	v5 adds set/policy/plan refs	future model update without header fork
batch-centric certificate	batch cert remains hot path; set-centric cert introduced	migration path と generic architecture
manual/operator registry assumptions	fork-local on-chain registry view	reorg-safe state
all-open mint target	active set + nonzero share + exact governing refs	model lifecycle gate
PCPB design	security semantics fixed, wiring fenced	unsafe early activation 回避
reviewed code status: algo-4 off	testnet-20 algo-4 active	public no-value rehearsal 進展
reviewed code status: registry absent	v5 registry active	V4 central change

REFERENCES

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar, “PHANTOM and GHOSTDAG: A Scalable Generalization of Nakamoto Consensus,” 2018–2021.
- [2] Kaspas Research and rusty-kaspa contributors, “rusty-kaspa: BlockDAG full-node implementation and protocol documentation.”
- [3] National Institute of Standards and Technology, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024.
- [4] J.-P. Aumasson et al., “BLAKE2: simpler, smaller, fast as MD5,” 2013.
- [5] NEAR Protocol contributors, “Borsh: Binary Object Representation Serializer for Hashing.”
- [6] *MISAKA Whitepaper Version 3.0: PQ BlockDAG, Native EVM, Canonical Integer AI Work*, 2026.
- [7] *MisakaLLM code snapshot and ADR collection*, snapshot dated 2026-08-01.